البحث المخصص للكور

407

W.P.F

W.P.F Research

FOR CORE 407

10-11-2015
DATE

2:08 PM
TIME

كريم رفعت

Introduction to WPF
Windows Presentation Foundation

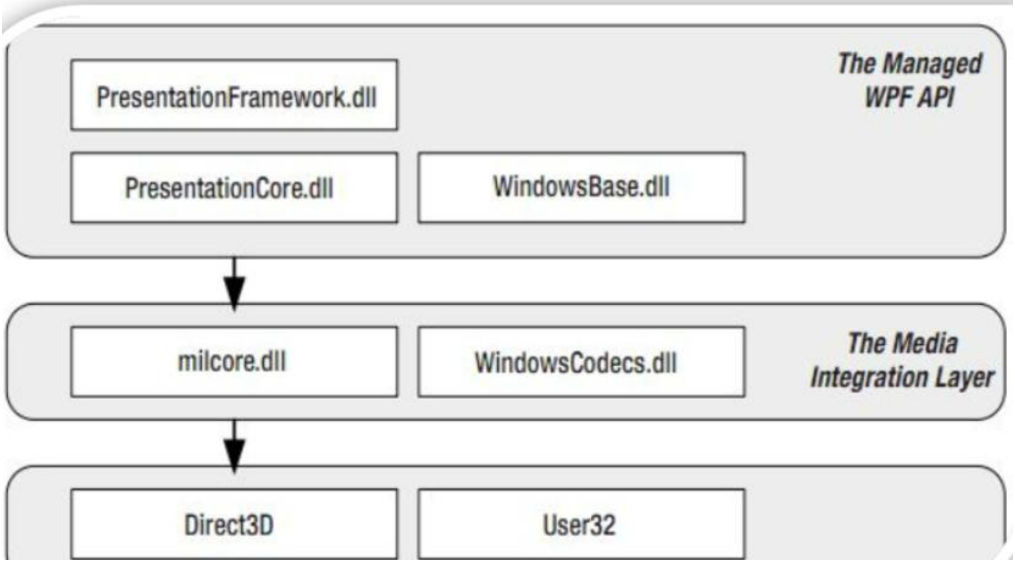
مميزات نظام WPF

WPF: A Higher-Level API

- WPF actually includes a basket of high-level services designed for application programmers.
- A web-like layout model.
- A rich drawing model.
- Animation as a first-class programming concept
- Support for audio and video media.
- Styles and templates.
- Commands.
- Declarative user interface.
- Page-based applications.

هيكل مكتبات نظام WPF

The Architecture of WPF



مهام المكتبات المختلفة

PresentationFramework.dll

- This holds the top-level WPF types, including those that represent windows, panels, and other types of controls.
- It also implements higher-level programming abstractions such as styles.
- Most of the classes you'll use directly come from this assembly.

PresentationCore.dll

- This holds base types, such as UIElement and Visual, from which all shapes and controls derive.

WindowsBase.dll

- This holds even more basic ingredients that have the potential to be reused outside of WPF, such as DispatcherObject and DependencyObject.

milcore.dll

- Media Integration Layer (MIL).
- Its composition engine translates visual elements into the textures that Direct3D expects.

WindowsCodecs.dll

- This is a low-level API that provides imaging support.
- For example, processing, displaying, and scaling bitmaps and JPEGs.

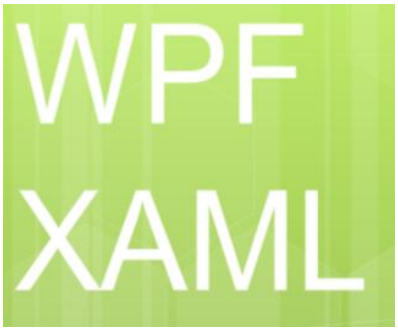
Direct3D

- This is the low-level API through which all the graphics in a WPF application are rendered.
- Direct3D renders all the drawing in WPF.

لغة الاكس اي ام ال

What is XAML?

- Short for Extensible Application Markup Language.
- Its primary role is to construct WPF user interfaces.
- It's unlikely that you'll write XAML by hand.
- Different ways to use XAML.
  - WPF XAML.
  - XPS XAML.
  - Silverlight XAML.
  - WF XAML(Windows Workflow Foundation).
- Its syntax like XML.

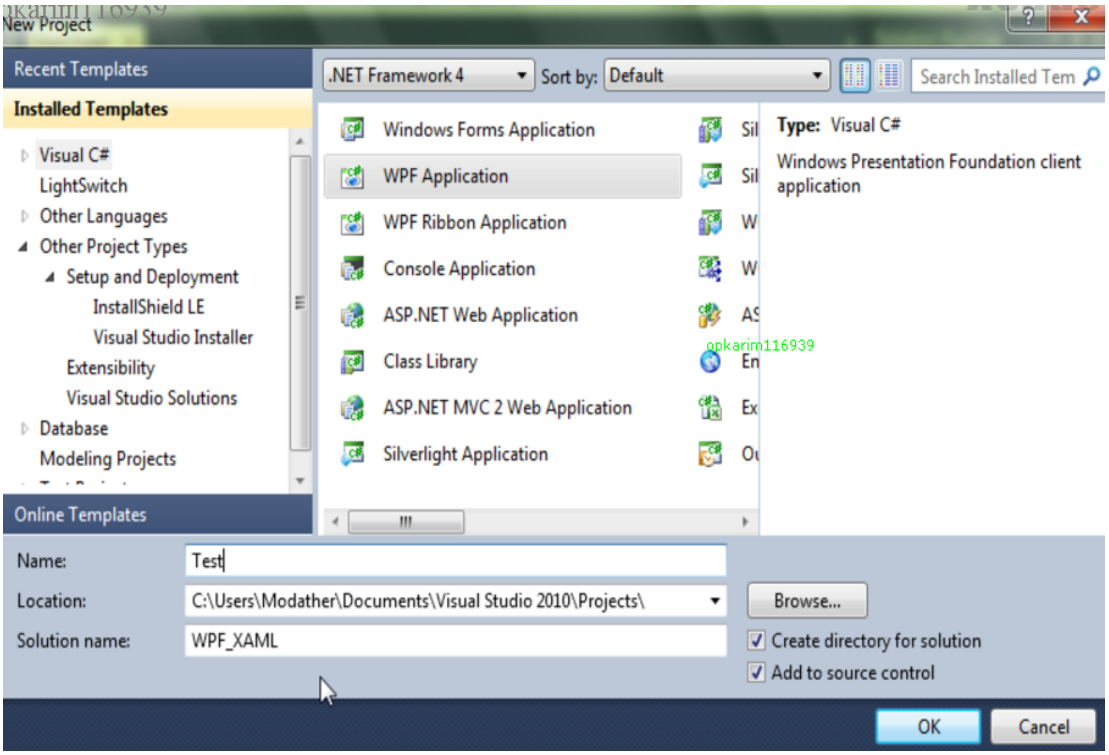


XAML Basics.

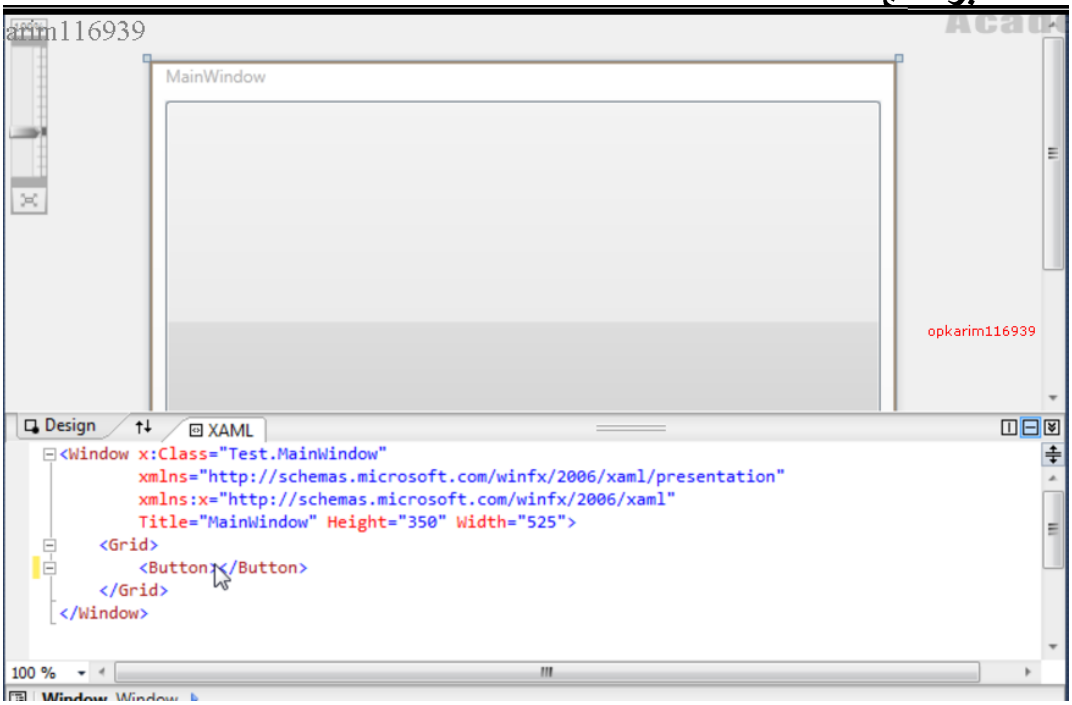
أساسيات الاي اكس ام ال

- Every XAML element is a .NET class.
  - EXP: <Button> is the "Button Class" in .NET.
- You can nest one element inside another like XML.
- You can set the properties of each class through attributes.
- In some situations an attribute isn't powerful enough to do the job. In these cases, you'll use nested tags with a special syntax.
- Anything you do by XAML you can do by C#.
- You can draw your interface without XAML code.

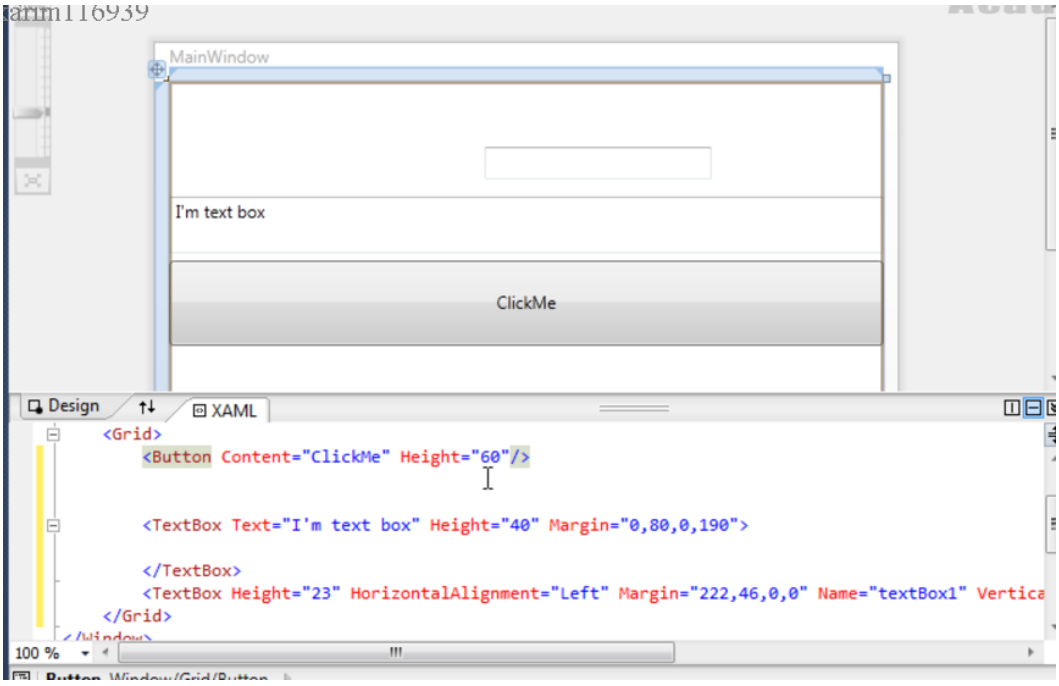
لإنشاء مشرع جديد



شكل شاشة البرنامج



كود اضافة مفتاح ومربع نص



XAML Basics(Cont.)

- Look at this XAML code

```
<Window x:Class="WpfApplication1.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="MainWindow" Height="350" Width="525">
    <Grid>

    </Grid>
</Window>
```

- This document includes only two elements—the top-level Window element, and the Grid, in which you can place all your controls.
- Top Level Elements
  - **Window – Page(Like the page in windows 7) – Application.**
- You could use only one Top-Level Element in your Application.
- Every element should be closed like that </Window> or by self-close
- Title="Window1" Height="300" Width="300">
- By this way you deal with Properties by setting attribute values (Title is Attribute).
- In this code “Red Color Code” we put the title of the window “Equals form in WFA” with “Window1”.
- The (Title) is an Attribute in XAML and Property in C# code.
- Here we make the window 300 by 300.

XAML Namespaces.

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

- The XAML parser also needs to know the .NET namespace where this class is located.
- The xmlns attribute is a specialized attribute in XML that’s reserved for declaring namespaces.
- (Presentation) It encompasses all the WPF classes.
- (xaml) includes various XAML utility features that allow you to influence how your document is interpreted.
- The namespace information allows the XAML parser to find the right class.

The Code-Behind Class

```
<Window x:Class="WpfApplication1.MainWindow"
```

- Code-Behind Class
  - The x prefix referred to the main class of the App.
  - This is a half-truth. So what is the real world?
    - This example creates a new class named MainWindow, which derives from the base Window class.
  - Partial classes allow you to split a class into two or more separate pieces for development and fuse them together in the compiled assembly.

اسماء العناصر

Naming Elements.

- Always you need to access your elements created in XAML doc.
- You may change properties or attach events.
- If you want to do that you should add name attribute besides your element.
- The ways to set name attribute
  - Form VS designer or XAML-Doc R-Click->Properties on the element then set the name value.
  - Beside element tag write x:Name="value" OR Name="value".
  - (<Grid x:Name="grid1">) = (private System.Windows.Controls.Grid grid1)

خصائص العناصر

Properties in XAML.
Simple Properties & Type Converters.

- <TextBox Name="txtQuestion" HorizontalAlignment="Stretch" FontFamily="Verdana" FontSize="24" Foreground="Green" ... >
- Object properties can be any .NET type.
- XAML, like all XML-based languages, is case-sensitive.
- You can't substitute <button> for <Button>.
- Type converters usually aren't case-sensitive.
- Foreground="White" and Foreground="white" have the same result.

Complex Properties.

- XAML provides another option: property-element syntax. With property-element syntax, you add a child element with a name in the form Parent.
- **<Grid Name="grid1"> <Grid.Background> ... </Grid.Background></Grid>**
- The key that makes this work is the period (.)
- You can add another tag to instantiate a specific class.
- **<Grid Name="grid1">..<Grid.Background>**
  <LinearGradientBrush>..</LinearGradientBrush>..</Grid.Background></Grid>

Properties in XAML.
Complex Properties.(Cont.)

- It's not enough to simply create the Linear Gradient Brush you also need to specify the colors in that gradient.

```
<Grid Name="grid1">
  <Grid.Background>
    <LinearGradientBrush>
      <LinearGradientBrush.GradientStops>
        <GradientStop Offset="0.00" Color="Red" />
        <GradientStop Offset="0.50" Color="Indigo" />
        <GradientStop Offset="1.00" Color="Violet" />
      </LinearGradientBrush.GradientStops>
    </LinearGradientBrush>
  </Grid.Background>
  ...
</Grid>
```

XAML

```
LinearGradientBrush brush = new LinearGradientBrush();

GradientStop gradientStop1 = new GradientStop();
gradientStop1.Offset = 0;
gradientStop1.Color = Colors.Red;
brush.GradientStops.Add(gradientStop1);

GradientStop gradientStop2 = new GradientStop();
gradientStop2.Offset = 0.5;
gradientStop2.Color = Colors.Indigo;
brush.GradientStops.Add(gradientStop2);

GradientStop gradientStop3 = new GradientStop();
gradientStop3.Offset = 1;
gradientStop3.Color = Colors.Violet;
brush.GradientStops.Add(gradientStop3);

grid1.Background = brush;
```

C#

Properties in XAML.
Markup Extensions.

- You may want to set a property value to an object that already exists. Or you may want to set a property value dynamically by binding it to a property in another control.
- Markup extensions can be used in nested tags or in XML attributes, which is more common.
- XML: `<Button ... Foreground="{x:StaticSystemColors.ActiveCaptionBrush}" >`
- Nested Tag:

```
<Button ... >
  <Button.Foreground>
    <x:Static Member="SystemColors.ActiveCaptionBrush"></x:Static>
  </Button.Foreground>
</Button>
```

Attached Properties.

```
<TextBox ... Grid.Row="0">
  Are you Ok ?
</TextBox>

<Button ... Grid.Row="1">
  Get Answer
</Button>

<TextBox ... Grid.Row="2">
  Yah !! Sure. I'm fine.
</TextBox>
```

العناصر المتداخلة

Nesting Elements.

- IList() -> Add()
- IDictionary() -> Add().
- ContentProperty -> uses child to set that property.

Properties in XAML.
Nesting Elements(Cont.)

- **XAML**
- <LinearGradientBrush>
- <LinearGradientBrush.GradientStops>
- <GradientStop Offset="0.00" Color="Red" />
- </LinearGradientBrush.GradientStops>
- </LinearGradientBrush>

- **C#**
- GradientStop gradientStop1 = new GradientStop();
- gradientStop1.Offset = 0;
- gradientStop1.Color = Colors.Red;
- IList list = brush.GradientStops;
- list.Add(gradientStop1);

- Note: May the collection defaults is null, you need to include the

Properties in XAML.
Nesting Elements(Cont.)

- Nested content doesn't always indicate a collection.
- <TextBox>content</TextBox>
- The Grid control isn't a collection and so it doesn't implement IList or IDictionary.
- Grid -> ContentProperty -> Applied to a panel -> panel.Children return UIElementCollection -> which implements IList.
- TextBox -> ContentProprety -> Text
- Button -> ContentProperty -> Content

Properties in XAML.
Nesting Elements(Cont.)

- As a general rule, all controls that derive from ContentControl allow a single nested element.
- All controls that derive from ItemsControl allow a collection of items that map to some part of the control (such as a list of items or a tree of nodes).
- All controls that derive from Panel are containers that are used to organize groups of controls.
- The ContentControl, ItemsControl, and Panel base classes all use the ContentProperty attribute.

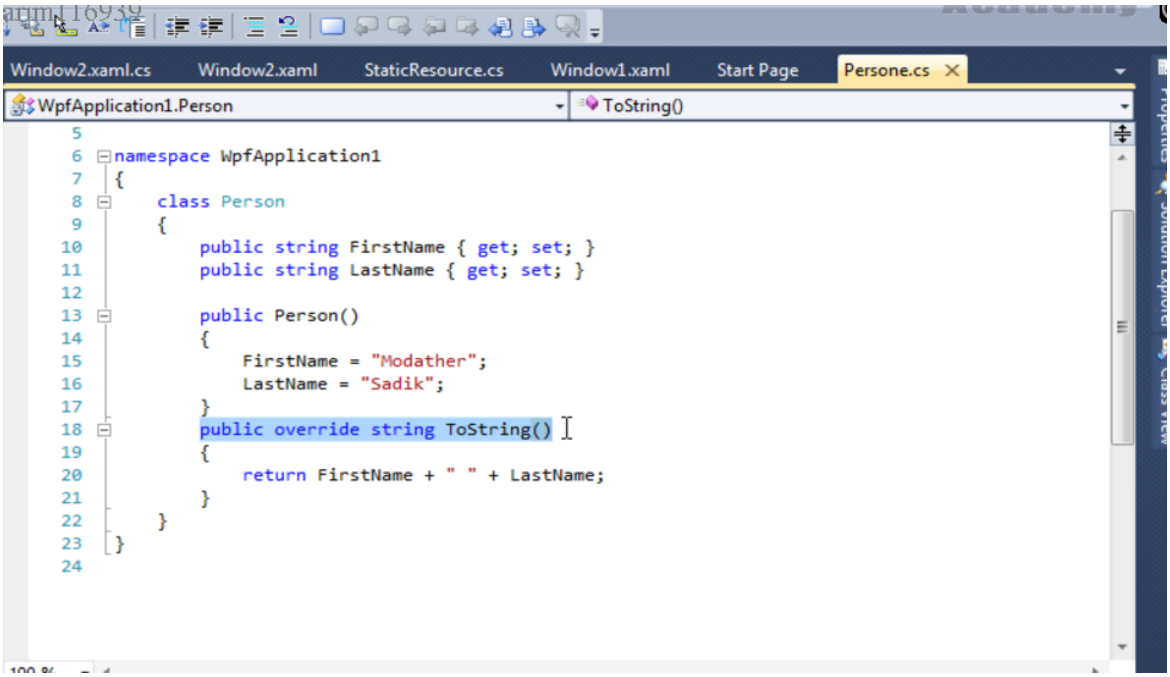
الوصول الي الخصائص عن طريق الاوبجكت

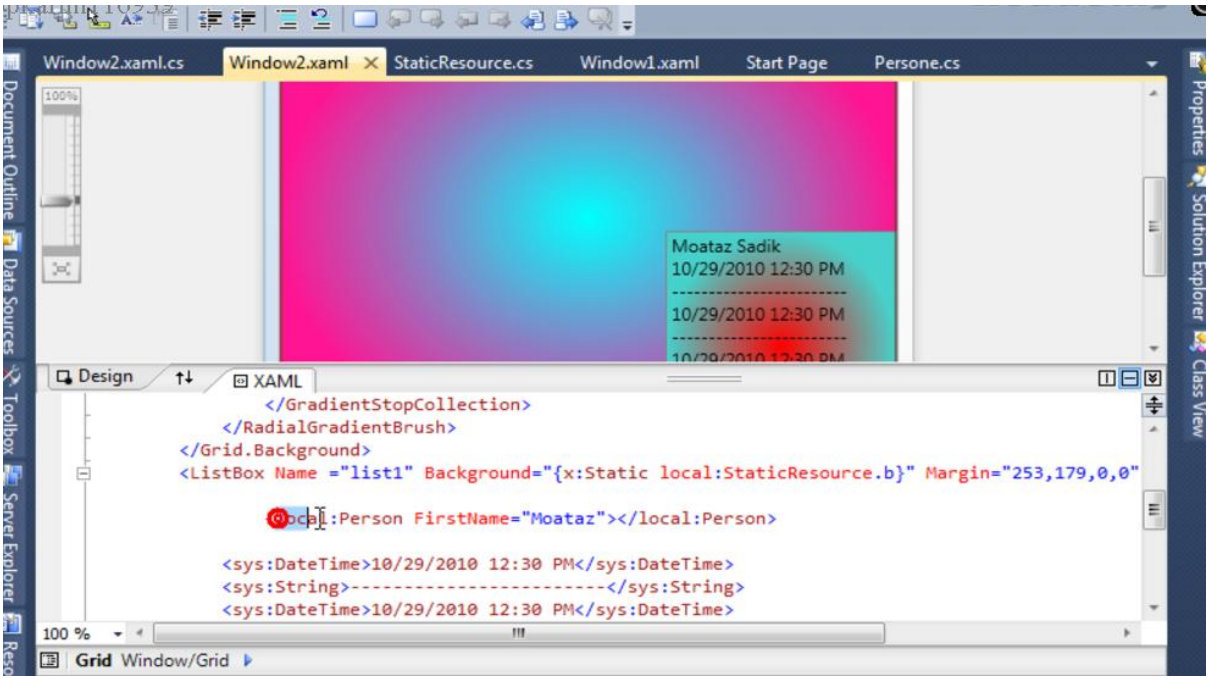
XAML 2009.
Advanced Object Creation.

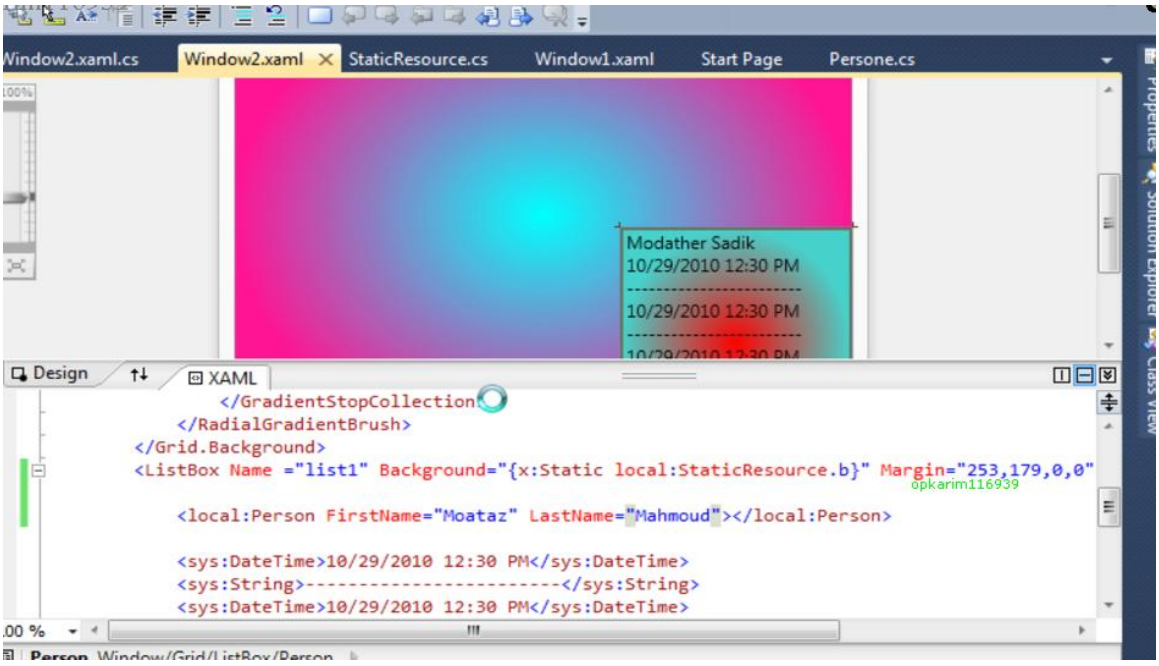
```
1.  <local:Person FirstName="Moataz"></local:Person>

2.  class Person
3.    {
4.      public string FirstName { get; set; }
5.      public string LastName { get; set; }

6.      public Person()
7.      {
8.        FirstName = "Modather";
9.        LastName = "Sadik";
10.     }
11.     public override string ToString()
12.     {
13.       return FirstName + " " + LastName;
14.     }
15.   }
```


Window2.xaml.cs
Window2.xaml
StaticResource.cs
Window1.xaml
Start Page
Persone.cs
WpfApplication1.Person
ToString()
namespace WpfApplication1
{
class Person
{
public string FirstName { get; set; }
public string LastName { get; set; }
public Person()
{
FirstName = "Modather";
LastName = "Sadik";
}
public override string ToString()
{
return FirstName + " " + LastName;
}
}
}


Window2.xaml.cs
Window2.xaml
StaticResource.cs
Window1.xaml
Start Page
Persone.cs
Moataz Sadik
10/29/2010 12:30 PM
10/29/2010 12:30 PM
Design
XAML
</GradientStopCollection>
</RadialGradientBrush>
</Grid.Background>
<ListBox Name ="list1" Background="{x:Static local:StaticResource.b}" Margin="253,179,0,0"
local:Person FirstName="Moataz"></local:Person>
<sys:DateTime>10/29/2010 12:30 PM</sys:DateTime>
<sys:String>----------</sys:String>
<sys:DateTime>10/29/2010 12:30 PM</sys:DateTime>
Grid Window/Grid





Understanding Layout(Cont.)

- A WPF window can hold only a single element.
- The reason is that Window class derived from ContentControl.
- To Add in more than one element, you need to place a container and then add other elements.
- You already did that by adding the Grid.

Containers Behavior

- Elements grow to fit their content.
- Element arranged by their container based on their size.
- Layout containers “share” the available space among their children.

opkarim116939

- Layout containers can be nested.

The Layout Containers

- All the WPF layout containers are panels.
- They derive from the abstract **System.Windows.Controls.Panel** class.
- The three public properties of Panel.
  - Background: The brush that’s used to paint the background.
  - Children: The collection of items that’s stored in the panel.
  - IsItemsHost: A Boolean value that’s true if the panel is showing the items that are associated with an ItemsControl.

Layout Properties

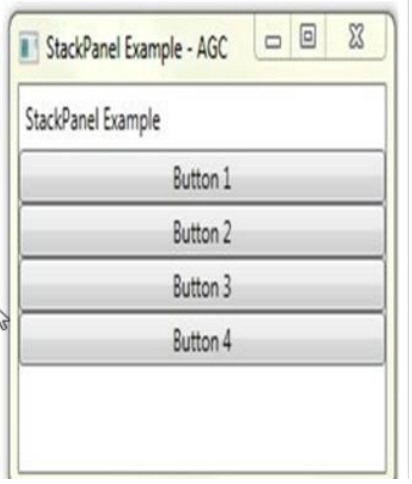
- HorizontalAlignment
- VerticalAlignment
- Margin
- MinWidth and MinHeight
- MaxWidth and MaxHeight
- Width and Height

Core Layout Panels

- StackPanel
- WrapPanel
- DockPanel
- Grid
- UniformGrid
- Canvas

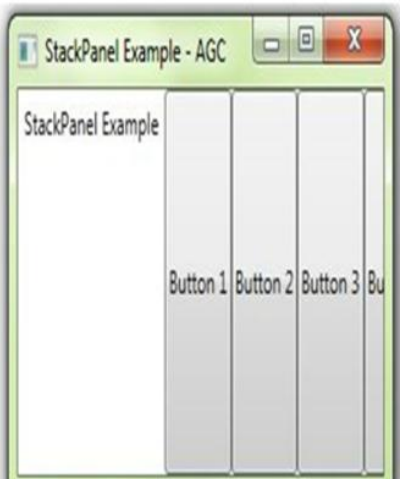
StackPanel

```
<Window x:Class="WpfApplication1.Window3"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel Example - AGC" Height="300" Width="300">
        <StackPanel>
            <Label>StackPanel Example</Label>
            <Button>Button 1</Button>
            <Button>Button 2</Button>
            <Button>Button 3</Button>
            <Button>Button 4</Button>
        </StackPanel>
</Window>
```



StackPanel Orientation

```
<Window x:Class="WpfApplication1.Window3"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel Example - AGC" Height="300" Width="300">
        <StackPanel Orientation="Horizontal">
            <Label>StackPanel Example</Label>
            <Button>Button 1</Button>
            <Button>Button 2</Button>
            <Button>Button 3</Button>
            <Button>Button 4</Button>
        </StackPanel>
</Window>
```



StackPanel Alignment

```
<Window x:Class="WpfApplication1.StackPanel_Alignment"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel-Alignment-AGC" Height="300" Width="300">
    <StackPanel>
        <Label HorizontalAlignment="Center">A Button Stack</Label>
        <Button HorizontalAlignment="Left">Button 1</Button>
        <Button HorizontalAlignment="Right">Button 2</Button>
        <Button>Button 3</Button>
        <Button>Button 4</Button>
    </StackPanel>
</Window>
```



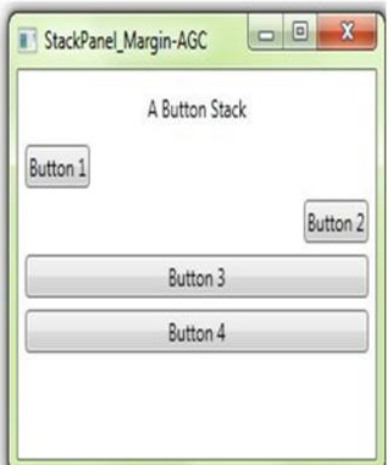
- The default HorizontalAlignment is Left for a label and Stretch for a Button.
- The StackPanel also has its own HorizontalAlignment and

StackPanel Margin

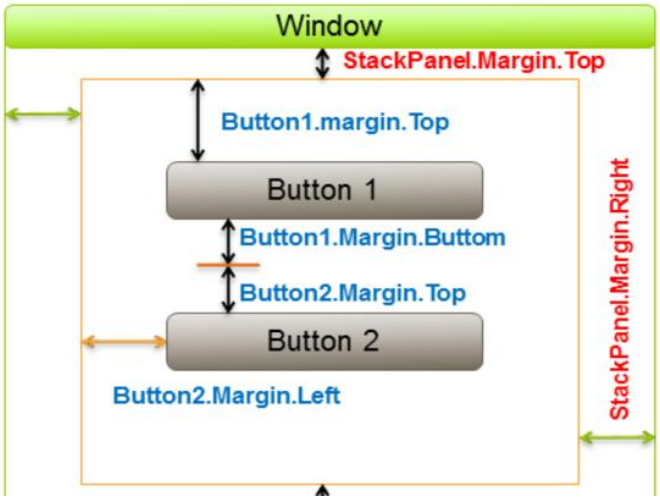
- Single margin for all sides
  - `<Button Margin="5">Button 1</Button>`
- Different margins for each side of a control in the order Left, Top, Right, Bottom
  - `<Button Margin="5,4,9,7">Button1</Button>`
- C# Code
  - `btn.Margin = new Thickness( 5 ); // Single`
  - `btn.Margin = new Thickness(5,4,9,7); // Different`

StackPanel Margin(Cont.)

```
<Window x:Class="WpfApplication1.StackPanel_Margin"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel_Margin-AGC" Height="300" Width="300">
    <StackPanel>
        <StackPanel Margin="3">
            <Label Margin="3" HorizontalAlignment="Center">
                A Button Stack</Label>
            <Button Margin="3" HorizontalAlignment="Left">Button 1</Button>
            <Button Margin="3" HorizontalAlignment="Right">Button 2</Button>
            <Button Margin="3">Button 3</Button>
            <Button Margin="3">Button 4</Button>
        </StackPanel>
    </StackPanel>
</Window>
```



StackPanel Margin(Cont.)



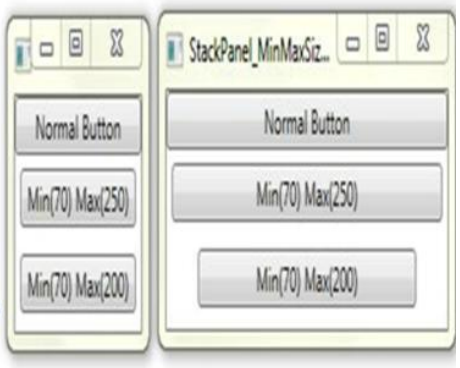
StackPanel Minimum & Maximum Size

- **When the StackPanel sizes a button, it considers several information:**

- Minimum size.
- Maximum size.
- The content.
- The size of the container.
- The horizontal alignment.

StackPanel Minimum & Maximum Size(Cont.)

```
<Window x:Class="WpfApplication1.StackPanel_MinMaxSize"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel_MinMaxSize-AGC" Height="300" Width="300">
    <StackPanel>
        <Button>Normal Button</Button>
        <Button Margin="5" MinWidth="70" MaxWidth="250">Min(70) Max(250)</Button>
        <Button  Margin="5" MinWidth="70" MaxWidth="200">Min(70) Max(200)</Button>
    </StackPanel>
</Window>
```



The Border

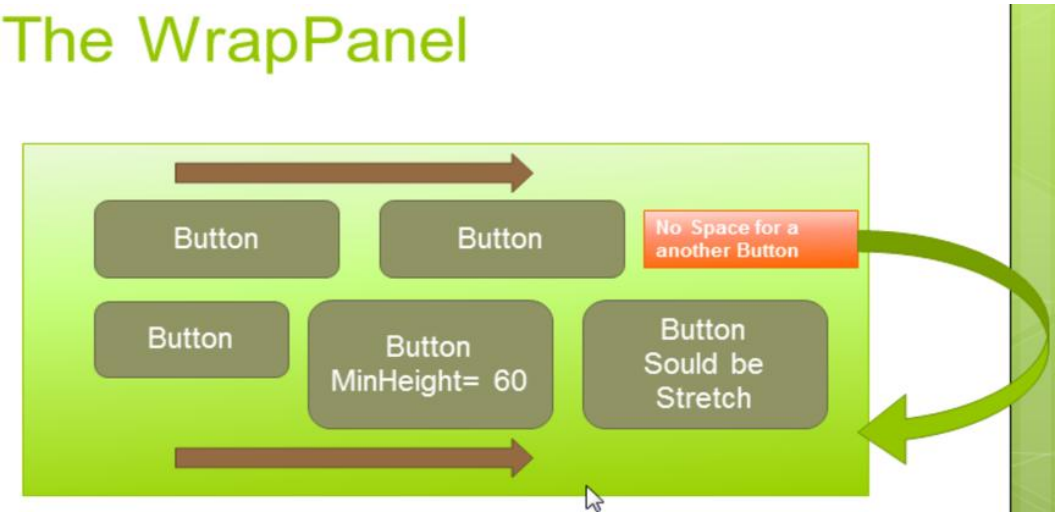
```
<Window x:Class="WpfApplication1.Border_AGC"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Border_AGC" Height="300" Width="300">
    <Border BorderBrush="YellowGreen" BorderThickness="8" CornerRadius="5"
            Padding="10" Margin="4" VerticalAlignment="Top" >
        <StackPanel>
            <Button Margin="3">Button 1</Button>
            <Button Margin="3">Button 2</Button>
            <Button Margin="3">Button 3</Button>
        </StackPanel>
    </Border>
</Window>
```



The Border(Cont.)

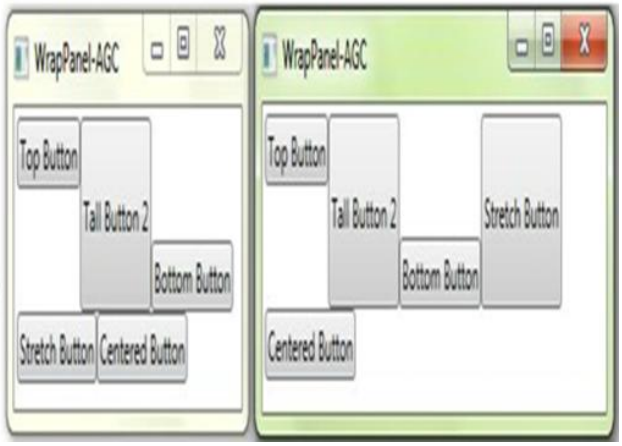
- The Border is a decorator, which is a type of element that's typically used to add some sort of graphical around an object.
- All decorators derive from the System.Windows.Controls.Decorator class.
- Most decorators are designed for use with specific controls.
- The Button uses a ButtonChrome decorator, while the ListBox uses the ListBoxChrome decorator.
- There are also two more general the Border discussed here and the Viewbox

The WrapPanel



The WrapPanel(Cont.)

```
<WrapPanel Margin="3">
    <Button VerticalAlignment="Top">Top Button</Button>
    <Button MinHeight="60">Tall Button 2</Button>
    <Button VerticalAlignment="Bottom">Bottom Button</Button>
    <Button>Stretch Button</Button>
    <Button VerticalAlignment="Center">Centered Button</Button>
</WrapPanel>
ndow>
```



The DockPanel(Cont.)

```
<Window x:Class="WpfApplication1.DockPanel"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DockPanel-AGC" Height="300" Width="300">
    <DockPanel LastChildFill="True">
        <Button DockPanel.Dock="Top">Top Button</Button>
        <Button DockPanel.Dock="Bottom">Bottom Button</Button>
        <Button DockPanel.Dock="Left">Left Button</Button>
        <Button DockPanel.Dock="Right">Right Button</Button>
        <Button>Remaining Space</Button>
    </DockPanel>
</Window>
```



The DockPanel(Cont.)

```
<Window x:Class="WpfApplication1.DockPanel2"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DockPanel2-AGC" Height="300" Width="300">
    <DockPanel LastChildFill="True">
        <Button DockPanel.Dock="Top">A Stretched Top Button</Button>
        <Button DockPanel.Dock="Top" HorizontalAlignment="Center">
            A Centered Top Button</Button>
        <Button DockPanel.Dock="Top" HorizontalAlignment="Left">
            A Left-Aligned Top Button</Button>
        <Button DockPanel.Dock="Bottom">Bottom Button</Button>
        <Button DockPanel.Dock="Left">Left Button</Button>
        <Button DockPanel.Dock="Right">Right Button</Button>
        <Button>Remaining Space</Button>
    </DockPanel>
</Window>
```



Nesting Layout Containers

```
<Window x:Class="WpfApplication1.NestingContainers"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="NestingContainers-AGC" Height="300" Width="300">
    <Border BorderBrush="Bisque" BorderThickness="5" Margin="6">
    <DockPanel LastChildFill="True">
        <StackPanel DockPanel.Dock="Bottom" HorizontalAlignment="Right"
   Orientation="Horizontal">
            <Button Margin="10,10,2,10" Padding="3">OK</Button>
            <Button Margin="2,10,10,10" Padding="3">Cancel</Button>
        </StackPanel>
        <TextBox DockPanel.Dock="Top" Margin="10">
            Nesting Layout Containers Test.</TextBox>
    </DockPanel>
    </Border>
</Window>
```



The Grid

- The Grid separates elements into an invisible grid of rows and columns.
- Although the Grid is designed to be invisible, you can set the Grid.ShowGridLines property to true to take a closer look.
- The Grid fits elements into predefined rows and columns.
- To add rows and columns you should use RowDefinition and ColumnDefinition objects.

The Grid(Cont.)

```
<Window x:Class="WpfApplication1.Grid1"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Grid1-AGC" Height="300" Width="300">
    <Grid ShowGridLines="True">
        <Grid.RowDefinitions>
            <RowDefinition></RowDefinition>
            <RowDefinition></RowDefinition>
            <RowDefinition></RowDefinition>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition></ColumnDefinition>
            <ColumnDefinition></ColumnDefinition>
            <ColumnDefinition></ColumnDefinition>
        </Grid.ColumnDefinitions>
        <Button>Top Left</Button>
        <Button Grid.Row="1" Grid.Column="1">Middle Center</Button>
        <Button Grid.Row="2" Grid.Column="2">Bottom Right</Button>
    </Grid>
```



The Grid
Fine-Tuning Rows and Columns

- The Grid supports three sizing strategies
  - Absolute sizes.(Fixed size)
  - Automatic sizes.
  - Proportional sizes.
- You can mix these different sizing modes.

The Grid
Fine-Tuning Rows and Columns(Cont.)

- Absolute sizes
  - `<ColumnDefinition Width="80"></ColumnDefinition>`
- Automatic sizes.
  - `<ColumnDefinition Width="Auto"></ColumnDefinition>`
- Proportional sizes
  - `<ColumnDefinition Width="*"></ColumnDefinition>`
- Look At this

```
1.  <RowDefinition Height="*"></RowDefinition>
2.  <RowDefinition Height="2*"></RowDefinition>
```

➢ This tells the Grid that the height of the second row should be twice the height of the first row.

The Grid
Fine-Tuning Rows and Columns(Cont.)

```
<Window x:Class="WpfApplication1.GridSizing"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="GridSizing-AGC" Height="300" Width="300">
    <Grid ShowGridLines="True">
        <Grid.RowDefinitions>
            <RowDefinition Height="*"></RowDefinition>
            <RowDefinition Height="Auto"></RowDefinition>
        </Grid.RowDefinitions>
        <TextBox Margin="10" Grid.Row="0">This is a test.</TextBox>
        <StackPanel Grid.Row="1"
                    HorizontalAlignment="Right" Orientation="Horizontal">
            <Button Margin="10,10,2,10" Padding="3">OK</Button>
            <Button Margin="2,10,10,10" Padding="3">Cancel</Button>
        </StackPanel>
    </Grid>
</Window>
```



You can use LayoutRounding to remove blur effect of shapes in the grid.

The Grid
Spanning Rows and Columns

- Make an element stretch over several cells.
- `<TextBox Margin="10" Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="3“/>`
- Here the textbox will take (0,0 – 0,1 – 0,2)

The Grid
Split Windows

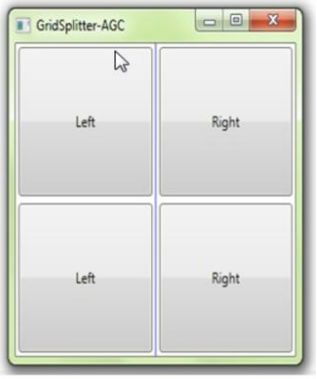
- The GridSplitter must be placed in a Grid cell.
- The GridSplitter always resizes entire rows or columns (not single cells).
- The GridSplitter is invisibly small.
- The GridSplitter alignment also determines.

The Grid
opkarim116939
Split Windows(Cont.)

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition></RowDefinition>
        <RowDefinition></RowDefinition>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition MinWidth="100"></ColumnDefinition>
        <ColumnDefinition Width="Auto"></ColumnDefinition>
        <ColumnDefinition MinWidth="50"></ColumnDefinition>
    </Grid.ColumnDefinitions>

    <Button Grid.Row="0" Grid.Column="0" Margin="3">Left</Button>
    <Button Grid.Row="0" Grid.Column="2" Margin="3">Right</Button>
    <Button Grid.Row="1" Grid.Column="0" Margin="3">Left</Button>
    <Button Grid.Row="1" Grid.Column="2" Margin="3">Right</Button>

    <GridSplitter Grid.Row="0" Grid.Column="1" Grid.RowSpan="2"
Width="1" VerticalAlignment="Stretch" HorizontalAlignment="Center"
ShowsPreview="False" BorderBrush="Blue" BorderThickness="1"></GridSplitter>
</Grid>
```



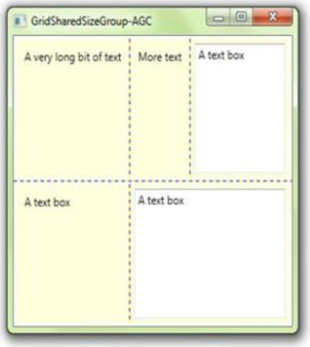
The Grid
Shared Size Groups

```
<Grid Grid.IsSharedSizeScope="True"  ShowGridLines="True">
    <Grid.RowDefinitions>
        <RowDefinition></RowDefinition>
        <RowDefinition></RowDefinition>
    </Grid.RowDefinitions>
    <Grid Margin="3" Background="LightYellow" ShowGridLines="True" Grid.Row="0">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" SharedSizeGroup="TextLabel"></ColumnDefinition>
            <ColumnDefinition Width="Auto"></ColumnDefinition>
            <ColumnDefinition></ColumnDefinition>
        </Grid.ColumnDefinitions>

        <Label Margin="5">A very long bit of text</Label>
        <Label Grid.Column="1" Margin="5">More text</Label>
        <TextBox Grid.Column="2" Margin="5">A text box</TextBox>
    </Grid>

    <Grid Margin="3" Background="LightYellow"  ShowGridLines="True" Grid.Row="1">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" SharedSizeGroup="TextLabel"></ColumnDefinition>
            <ColumnDefinition></ColumnDefinition>
        </Grid.ColumnDefinitions>

        <Label Margin="5" Name="lbl">Short</Label>
        <TextBox Grid.Column="1" Margin="5" TextChanged="TextBox_TextChanged">A text box</TextBox>
    </Grid>
</Grid>
```



The UniformGrid

- The UniformGrid. Unlike the Grid, the UniformGrid doesn't require (or even support) predefined columns and rows.
- you simply set the Rows and Columns properties to set its size.
- Each cell is always the same size because the available space is divided equally.
- elements are placed into the appropriate cell based on the order in which you define them.
- There are no attached Row and Column properties

The UniformGrid

```
<Window x:Class="WpfApplication1.UniformGrid"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="UniformGrid-AGC" Height="300" Width="300">

    <UniformGrid Rows="2" Columns="2">
        <Button>Top Left</Button>
        <Button>Top Right</Button>
        <Button>Bottom Left</Button>
        <Button>Bottom Right</Button>
    </UniformGrid>
</Window>
```

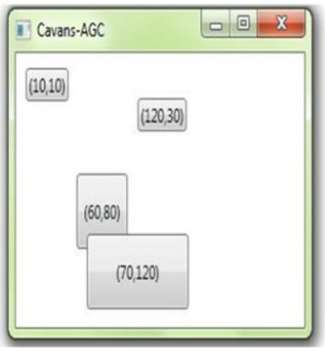


Canvas

- it's a valuable tool if you need to build something a little different (such as a drawing surface for a diagramming tool).
- The Canvas is also the most lightweight of the layout containers.

Canvas

```
<Window x:Class="WpfApplication1.CavansExp"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Cavans-AGC" Height="300" Width="300">
    <Canvas>
        <Button Canvas.Left="10" Canvas.Top="10">(10,10)</Button>
        <Button Canvas.Left="120" Canvas.Top="30">(120,30)</Button>
        <Button Canvas.Left="60" Canvas.Top="80" Width="50" Height="50">
            (60,80)</Button>
        <Button Canvas.Left="70" Canvas.Top="120" Width="100" Height="50">
            (70,120)</Button>
    </Canvas>
</Window>
```



ZIndex is used to make the element overtop other elements.

WPF Dependency Properties & Routed Events

Understanding Dependency Properties

- The dependency property is a special type of properties that have the following features:
  - Set and Get Values.
  - Notification system when the value changed.
  - Used in animation, Data Binding, Styles.
  - There is a way to make it an attached property.
  - You can share it with another element or object.
  - Get a property with inherited default value.

You can add dependency properties only to dependency objects classes that derive from DependencyObject.

Defining a Dependency Property

- The Class must derived from *DependencyObject* class.

```
1. public class Person:DependencyObject
2. {
3.     // Some code here
4. }
```

Defining a Dependency Property(Cont.)

- First Step. Define an object that represents your property.

```
1.  public class Person:DependencyObject
2.  {
3.      public static readonly DependencyProperty LastNameProperty;
4.  }
```

  - The field is defined with the *readonly* keyword, which means it can be set only in the static constructor.
  - The information about your property needs to be available all the time, so your DependencyProperty object must be defined as a static field.

The field that defines a dependency property has the name of the ordinary property, plus the word Property at the end.

Defining a Dependency Property(Cont.)

- Second Step. Registering a Dependency Property.
  - The only way to make a new object from DependencyProperty class is by Register() method.
  - The Register() method needs some argument.
    - The name of the property.
    - The type of the property.
    - The type of the property owner.
    - Optionally. The property metadata settings.
      - The metadata includes the default value and two methods one to correct the property value and the second to fires when the value changed.
    - Optionally. Method to validate the value.

Defining a Dependency Property(Cont.)

- Second Step(Cont.).

```
class Person:DependencyObject{

1.  public static readonly DependencyProperty LastNameProperty =
    DependencyProperty.Register("LastName" , typeof(string)
2.   , typeof(Persone) , metadata , ValidateCallback);


3.  public static PropertyMetadata metadata = new
4.  PropertyMetadata("Ali", ChangedCallback , CoerceCallback);
```

Defining a Dependency Property(Cont.)

- Second Step(Cont.).

```
1. private static void ChangedCallback(DependencyObject sender ,
   DependencyPropertyChangedEventArgs e)
2. {// Fires when the value of the property changed.}

3. private static object CoerceCallback(DependencyObject sender ,
   object o)
4. {// Correct the value of the property.}

5. private static bool ValidateCallback(object value)
6. {// Validate the value of the property.}}
```

Defining a Dependency Property(Cont.)

- Last Step. Adding a Property Wrapper.

```
1.  public string LastName
2.  {
3.    set{this.SetValue( LastNameProperty , value );}

4.    get{return ( string )this.GetValue( LastNameProperty );}
5.  }
```

- Setting value into the property must call *SetValue()* method and getting value must use *GetValue()* method.
- When you create the property wrapper, you should include nothing more than a call to *SetValue()* and a call to *GetValue()*.

Using Dependency Property

- **C#**

```
1.  Person obj = new Person();
2.  obj.LastName = new string(new char[]{ 'A' , 'l' , 'i' });
3.  obj.LastName = "Ali";
```

- **XAML**

```
1. <local:Persone x:Name="obj" LastName="Ali"></local:Persone>
```

Four-step process to determine a property value.

1. Determine the base value from:
   1. The default value (as set by the *PropertyMetadata* object).
   2. The inherited value (if the *PropertyMetadata.Inherits* a value).
   3. The value from a theme style.
   4. The value from a project style.
   5. The local value (a value you've set directly on this object using code or XAML)
2. If the property is set using an expression, evaluate that expression "*data binding* and *resources*".
3. If this property is the target of animation, apply it.
4. Run the *CoerceValueCallback* to "correct" the value.

Shared Dependency Properties

- Some classes share the same dependency property

```
1.  TextBlock.FontFamilyProperty = TextElement
    .FontFamilyProperty.AddOwner( typeof( TextBlock ) );
```

  - They shared the property even though they have separate class hierarchies.
  - You can use an overload of the *AddOwner()* method that allows you to supply a validation callback and a new *PropertyMetadata*.

Understanding Routed Events

- Routed events allow an event to be handled on one element (such as a label) even though it originates on another (such as an image inside that label).

Defining, Registering, and Wrapping a Routed Event

```
1.  public abstract class ButtonBase : ContentControl{
2.   // The event definition.
3.   public static readonly RoutedEvent ClickEvent;
4.   // The event registration.
5.   static ButtonBase(){
6.   ButtonBase.ClickEvent = EventManager.RegisterRoutedEvent(
    "Click",RoutingStrategy.Bubble,typeof(RoutedEventHandler),
    typeof(ButtonBase)); }
7.   // The event wrapper.
8.   public event RoutedEventHandler Click{
9.    add
10.   { base.AddHandler( ButtonBase.ClickEvent , value );}
11.   remove
12.   { base.RemoveHandler( ButtonBase.ClickEvent , value ); } } }
```

Sharing Routed Events

```
UIElement.MouseUpEvent =
Mouse.MouseUpEvent.AddOwner(typeof(UIElement));
```

Raising a Routed Event

```
1.  RoutedEventArgs e = new RoutedEventArgs(
    ButtonBase.ClickEvent , this );

2.  base.RaiseEvent( e );
```

Handling a Routed Event

- XAML
  - `<Button Click="Btn_Click">Click</Button>`
- C#

```
1.  btn.Click+(-)=new
    MouseButtonEventHandler(Btn_Click);
2.  void Btn_Click(object sender , RoutedEventArgs e)
3.  { // Do something }
```

- OR

```
1.  btn.Add(OR Remove)Handler ( Mouse.MouseDownEvent
    ,new MouseButtonEventHandler( Btn_Click ) );

2.  btn.AddHandler( UIElement.MouseDownEvent ,
3.  new MouseButtonEventHandler( Btn_Click ) );
```

Event Routing

- Routed events actually come in the following three flavors:
  - Direct events. These are like ordinary .NET events
    - Example: MouseEnter Event.
  - Bubbling events. These events travel up the containment hierarchy.
    - Example: MouseUp Event.
  - Tunneling events. These events travel down the containment hierarchy.
    - Example: PreviewKeyDown Event.

Event Routing
The RoutedEventArgs Class

- Source
  - The object that raised the event.
- OriginalSource
  - The object originally raised the event.

opkarim116939

- RoutedEvent
  - Information about event like name, owner type.
- Handled
  - Halt the event bubbling or tunneling process

Event Routing
Attached Events

- Many controls have their own specialized events.
- The button is one example it adds a Click event that isn't defined by any base class.
- If you want to handle all buttons inside a container you should use attached events.

```
<StackPanel Button.Click="AnButtonClicked" Margin="5">
  <Button Name="cmd1">Command 1</Button>
  <Button Name="cmd2">Command 2</Button>
  <Button Name="cmd3">Command 3</Button>
</StackPanel>
```

- If you want it Bubble Event you should use AddHandler() Method

Event Routing
Attached Events (cont.)

- In the Event code you could use some info to know which element fired the event.

```
1.  private void AnButtonClicked
2.  (object sender , RoutedEventArgs e)
3.          {
4.              if ( e.Source == cmd1 )
5.              { /* Do Something */ }
6.              else if ( e.Source == cmd2 )
7.              { /* Do Something */ }
8.              else if ( e.Source == cmd3 )
9.              { /* Do Something */ }
10.         }
```

WPF Events

- Lifetime events. These events occur when the element is initialized, loaded, or unloaded.
- Mouse events. These events are the result of mouse actions.
- Keyboard events. These events are the result of keyboard actions.
- Stylus events. These events are the result of using the pen-like stylus, which takes the place of a mouse on a Tablet PC.
- Multitouch events. These events are the result of touching down with one or more fingers on a multitouch screen. They're only supported in Windows 7.

WPF Events
Lifetime Events

- Initialized: Occurs after the element is instantiated and its properties have been set.
- Loaded: Occurs after the entire window has been initialized and styles and data binding have been applied.
- Unloaded: Occurs when the element has been released, either because the containing window has been closed or the specific element has been removed from the window.

WPF Events
Lifetime Events(cont.)

- ContentRendered: Occurs immediately after the window has been rendered for the first time.
- Activated: Occurs when the user switches to this window , Activated also fires when the window is loaded for the first time.
- Deactivated: Occurs when the user switches away from this window (for example, by moving to another window in your application or another application). Deactivated also fires when the window is closed by a user, after the Closing event but before Closed.
- Closing : Occurs when the window is closed, either by a user action or programmatically using the Window.Close() method or the Application.Shutdown() method.
- Closed: Occurs after the window has been closed. However, the element objects are still accessible, and the Unloaded event hasn't fired yet.

WPF Events
Input Events - Keyboard Input

Name	Routing	Description
PreviewKeyDown	Tunneling	Occurs when a key is pressed.
KeyDown	Bubbling	
PreviewTextInput	Tunneling	Occurs when a keystroke is complete and the element is receiving the text input. This event isn't fired for keystrokes that don't result in text.
TextInput	Bubbling	
PreviewKeyUp	Tunneling	Occurs when a key is released.
KeyUp	Bubbling	

- Handling a Key Press Example.
- What is Focus ?
- Getting Key State.

WPF Events
Input Events - Mouse Input

- MouseMove Event
  - Display the position of the mouse pointer(Example).
    - Use the e.GetPosition(this); Method.
- MouseClick Event
  - There are two events that react to the mouse wheel: PreviewMouseWheel and MouseWheel.
  - Some elements add higher-level mouse events. For example, the Control class adds PreviewMouseDoubleClick and MouseDoubleClick events that take the place of the MouseLeftButtonUp event. Similarly, the Button class raises a Click event that can be triggered by the mouse or keyboard.

WPF Events
Input Events - Mouse Input(cont.)

- MouseClick Events (cont.)

Name	Routing	Description
PreviewMouseLeftButtonDown and PreviewMouseRightButtonDown	Tunneling	Occurs when a mouse button is pressed
MouseLeftButtonDown and MouseRightButtonDown	Bubbling	
PreviewMouseLeftButtonUp and PreviewMouseRightButtonUp	Tunneling	Occurs when a mouse button is released
MouseLeftButtonUp and MouseRightButtonUp	Bubbling	

WPF Events
Input Events - Mouse Input(cont.)

- Drag-and-Drop
  - WPF changes very little about drag-and-drop operations.
  - the methods and events that are used for drag-and-drop operations are centralized in the System.Windows.DragDrop class and then used by other classes.

- Essentially, a drag-and-drop operation unfolds in three steps:
  - The user clicks an element (or selects a specific region inside it) and holds the mouse button down.
  - The user moves the mouse over another element.
  - When the user releases the mouse button, the element receives the information. The operation can be canceled by pressing the Esc key.

WPF Events
Input Events - Mouse Input(cont.)

- Drag-and-Drop(cont.)
  - Enable Drop Operation.

```
<Label  Grid.Row= "1"  AllowDrop="True" Drop="label2_Drop"> Drop here </Label>
```

  - Receive data.

```
( ( Label )sender ).Content = e.Data.GetData( DataFormats.Text );
```

- Some controls supports drag and drop by default like Textbox «Try it yourself».
- To make an element dragable you write this code in the DragEnter event.

```
1.    if ( e.Data.GetDataPresent( DataFormats.Text ) )
2.                  e.Effects = DragDropEffects.Copy;
3.              else
4.                  e.Effects = DragDropEffects.None;
```

WPF Events

Input Events - Mouse Input(cont.)

- Drag-and-Drop(cont.)
  - Enable Drop Operation.
    - `<Label Grid.Row= "1" AllowDrop="True" Drop="label2_Drop"> Drop here </Label>`
  - Receive data.
    - `( ( Label )sender ).Content = e.Data.GetData( DataFormats.Text );`

- Some controls supports drag and drop by default like Textbox «Try it yourself».
- To make an element dragable you write this code in the DragEnter event.

```
1.    if ( e.Data.GetDataPresent( DataFormats.Text ) )
2.            e.Effects = DragDropEffects.Copy;
3.        else
4.            e.Effects = DragDropEffects.None;
```

WPF Controls

WPF Controls

- **Content controls**. These controls can contain nested .
- **Headered content controls**. These are content controls that allow you to add a content and title.
- **Text controls**. These controls allow users to enter input.
- **List controls**. These controls show collections of items in a list.
- **Range-based controls**. These controls have just one thing in common: a Value property that can be set to any number in a prescribed range.
- **Date controls**. allow users to select dates.

The Control Class

- All controls derive from the System.Windows.Control class, adds:
  - The ability to set the alignment of content inside the control
  - The ability to set the tab order
  - Support for painting a background, foreground, and border
  - Support for formatting the size and font of text content

The Control Class Font Inheritance

- When you set any of the font properties, the values flow through to nested objects.
- If you set the FontFamily property for the top-level window, every control in that window gets the same FontFamily value (unless the control explicitly sets a different font).

The Control Class Font Substitution

- ```
  <Button FontFamily="Technical Italic, Comic Sans MS, Arial">
   A    Button</Button>
  ```
- Set Font Families to ListBox

```
1.  foreach ( FontFamily fontFamily in Fonts.SystemFontFamilies )
2.  {
3.    lstFonts.Items.Add( fontFamily.Source );
4.  }
```

The Control Class Font Embedding

- ```
  <Label FontFamily="./#Bayern" FontSize="20">
    This is an embedded font</Label>
  ```
- You need to make sure you get the font family name exactly right, and you need to make sure you choose the correct build action for the font file.
- Visual Studio doesn't currently provide design support for embedded fonts.

pkarim11@gmail.com

The Control Class
Text Formatting Mode

```
<TextBlock FontSize="12" Margin="5">
This is a Test. Ideal text is blurry at small sizes.
</TextBlock>

<TextBlock FontSize="12" Margin="5"
TextOptions.TextFormattingMode="Display">
This is a Test. Display text is crisp at small sizes.
</TextBlock>
```

The Control Class
Mouse Cursors

- C#: `this.Cursor = Cursors.Wait;`
- XAML: `<Button Cursor="Help">Help</Button>`
- Overrides the Cursor property of every element.
  - `Mouse.OverrideCursor = Cursors.Wait;`
  - To removecursor override, set the it to null.
- Example.

The Content Controls

The Content Controls Button

- It's derevied from ButtonBase calss.
- Common Properties.
  - **IsCancel:** Fired the button event when you press Escape key.
  - **IsDefault :** Fired the event when you press Enter key.
- Three more classes derive from ButtonBase.
  - GridViewColumnHeader.
  - RepeatButton.
  - ToggleButton.
- Example.

List Controls The ListBox

- The ListBox class allows multiple selection.
- To add items to the ListBox, you can nest ListBoxItem elements inside the ListBox element.
- ListBoxItem can hold one single element.
- You also can place your objects directly inside the ListBox element without using ListBoxItem.
- Example.

List Controls
The ComboBox

- The ComboBox is similar to the ListBox control. It holds a collection of ComboBoxItem objects.
- The ComboBox control uses a drop-down list, which means only one item can be selected at a time.
- One limitation of the ComboBox is the way it sizes itself when you use automatic sizing.
- there's no easy way to tell the ComboBox to take the size of its largest contained item.

Range-Based Controls

- WPF includes three controls that use the concept of a range.
  - ScrollBar, ProgressBar, and Slider.
- Properties of the RangeBase Class
  - Value.
  - Maximum , Minimum.
  - SmallChange.
  - LargeChange.



Introduction to Element Binding

- Data binding is a relationship that tells WPF to extract some information from a source object and use it to set a property in a target object.
- The target property is always a dependency property, and it's usually in a WPF element.
- The Element Binding is the relation between elements.
- when you change the value of the source object, the target object is updated immediately.

The Binding Expression

```
<ProgressBar
  Value="{Binding ElementName=SourceName,
  Path=PropertyName }"></ProgressBar>
```

- WPF doesn't raise exceptions to notify you about data binding problems.

Binding Modes

- One of features of binding is that your target is updated automatically, no matter how the source is modified.
- Example (Different Modification Ways).
- Values from the BindingMode Enumeration
  - OneWay.
  - TwoWay.
  - OneTime.
  - OneWayToSource.
  - Default.

Creating Bindings with Code

```
1.  Binding binding = new Binding();
2.  binding.Source = sliderFontSize;
3.  binding.Path = new PropertyPath( "Value" );
4.  binding.Mode = BindingMode.TwoWay;
5.  lblSampleText.SetBinding( TextBlock.FontSizeProperty ,
    binding );
```

- You can also remove a binding with code using two static methods of the BindingOperations class.
  - ClearBinding(), ClearAllBinding().

```
1.  BindingOperations.ClearAllBindings( lblSampleText );
```

Values from the UpdateSourceTrigger Enumeration

- PropertyChanged.
- LostFocus.
- Explicit.
- Default.

- C# Code

```
1.  // Get the binding that's applied to the text box.
2.  BindingExpression binding =
    txtFontSize.GetBindingExpression( TextBox.TextProperty );

3.  // Update the linked source (the TextBlock).
4.  binding.UpdateSource();
```

Binding Non Element Type

- When binding to an object that isn't an element, you need to give up the Binding.ElementName property and use one of the following properties instead:
  - Source.
  - RelativeSource.
  - DataContext.

Binding Non Element Type Source

- `<TextBlock Text="{Binding Source={x:Static SystemFonts.IconFontFamily}, Path=Source}"></TextBlock>`

- Using Static Resource

```
<Window.Resources>
<FontFamily x:Key="CustomFont">Arial</FontFamily>
</Window.Resources>

<TextBlock Text="{Binding Source={StaticResource CustomFont}, Path=Source}"></TextBlock>
```

Binding Non Element Type RelativeSource

- The RelativeSource property allows you to point to a source object based on its relation to the target object.
- Values from the RelativeSourceMode Enumeration
  - Self.
  - FindAncestor.
  - PreviousData.
  - TemplatedParent.

Binding Non Element Type RelativeSource(cont.)

- Syntax (Complex)

```
<TextBlock><TextBlock.Text>
<Binding Path="Title">
<Binding.RelativeSource>
<RelativeSource
 Mode="FindAncestor" AncestorType="{x:Type Window}" />
</Binding.RelativeSource> </Binding>
</TextBlock.Text> </TextBlock>
```

- Syntax (Simple)

```
<TextBlock Text=
"{Binding Path=Title, RelativeSource={RelativeSource FindAncestor AncestorType={x:Type Window}} }">
</TextBlock>
```

DataContext

- In some cases, you'll have a number of elements that bind to the same object.
- Putting the DataContext as Resource

```
<StackPanel
DataContext="{x:Static SystemFonts.IconFontFamily}">
```

- Binding to this source

```
<TextBlock Margin="5"
Text="{Binding Path=Source}"></TextBlock>
```

Binding Non Element Type RelativeSource(cont.)

- Syntax (Complex)

```
<TextBlock><TextBlock.Text>
<Binding Path="Title">
<Binding.RelativeSource>
<RelativeSource
 Mode="FindAncestor“ AncestorType="{x:Type Window}" />
</Binding.RelativeSource> </Binding>
</TextBlock.Text> </TextBlock>
```

- Syntax (Simple)

```
<TextBlock Text=
"{Binding Path=Title, RelativeSource={RelativeSource
FindAncestor AncestorType={x:Type Window}} }">
</TextBlock>
```

DataContext

- In some cases, you'll have a number of elements that bind to the same object.
- Putting the DataContext as Resource

```
<StackPanel
DataContext="{x:Static SystemFonts.IconFontFamily}">
```

- Binding to this source

```
<TextBlock Margin="5"
Text="{Binding Path=Source}"></TextBlock>
```

Introduction To Commands

- The WPF Command Model
  - Commands.
  - Command bindings.
  - Command sources.
  - Command targets.

The ICommand Interface

- The heart of the WPF command model is the System.Windows.Input.ICommand interface, which defines how commands work. This interface includes two methods and an event:

```
public interface ICommand
{
    void Execute(object parameter);
    bool CanExecute(object parameter);

    event EventHandler CanExecuteChanged;
}
```

The RoutedCommand Class

- When creating your own commands, you won't implement ICommand directly. Instead, you'll use the System.Windows.Input.RoutedCommand class, which implements this interface for you.

The RoutedUICommand Class

- Most of the commands you'll deal with won't be RoutedCommand objects; rather, they will be instances of the RoutedUICommand class, which derives from RoutedCommand.
- In fact, all the ready-made commands that WPF provides are RoutedUICommand objects.

The Command Library

- Commands are exposed through the static properties of five dedicated static classes:
  - ApplicationCommands.
  - NavigationCommands.
  - EditingCommands.
  - ComponentCommands.
  - MediaCommands.

opkarim116939

Executing Commands

- The RoutedUICommand doesn't have any hardwired functionality.
- It represents a command.
- To trigger this command, you need a command source.
- To respond to this command, you need a command binding that forwards execution to an event handler.

Executing Commands Command Sources

- The easiest way to trigger the commands in the command library is to hook them up to a control that implements the ICommandSource interface.
- Which includes controls that derive from ButtonBase (Button, CheckBox, and so on), individual ListBoxItem objects, the Hyperlink, and the MenuItem.

Executing Commands Command Sources

- Properties of the ICommandSource Interface
  - Command.
  - CommandParameter.
  - CommandTarget.

- `<Button Command="ApplicationCommands.New">New</Button>`

Executing Commands Command Bindings

- When you attach a command to a command source. The command source will be automatically disabled.
- To change this state, you need three things:
  - What to do when the command is triggered.
  - How to determine whether the command can be performed. optional
  - Where the command is in effect.
- You can add this code to the constructor of your window:

```
1.  // Create the binding.
2.  CommandBinding binding =
3.  new CommandBinding(ApplicationCommands.New );
4.  // Attach the event handler.
5.  binding.Executed += NewCommand_Executed;
6.  // Register the binding.
7.  this.CommandBindings.Add( binding );
```

Executing Commands
Command Bindings

- Binding Using XAML

```
<Window.CommandBindings>
<CommandBinding Command="ApplicationCommands.New"
 Executed="NewCommand_Executed"></CommandBinding>
</Window.CommandBindings>

<StackPanel Margin="5">
<Button Padding="5"
Command="ApplicationCommands.New">New</Button>
</StackPanel>
```

Invoking a Command Directly

- You can call a method directly from any event handler using the Execute() method.
- You need to pass in the parameter value (or a null reference) and a reference to the target element:
  - `ApplicationCommands.New.Execute( null , targetElement );`